

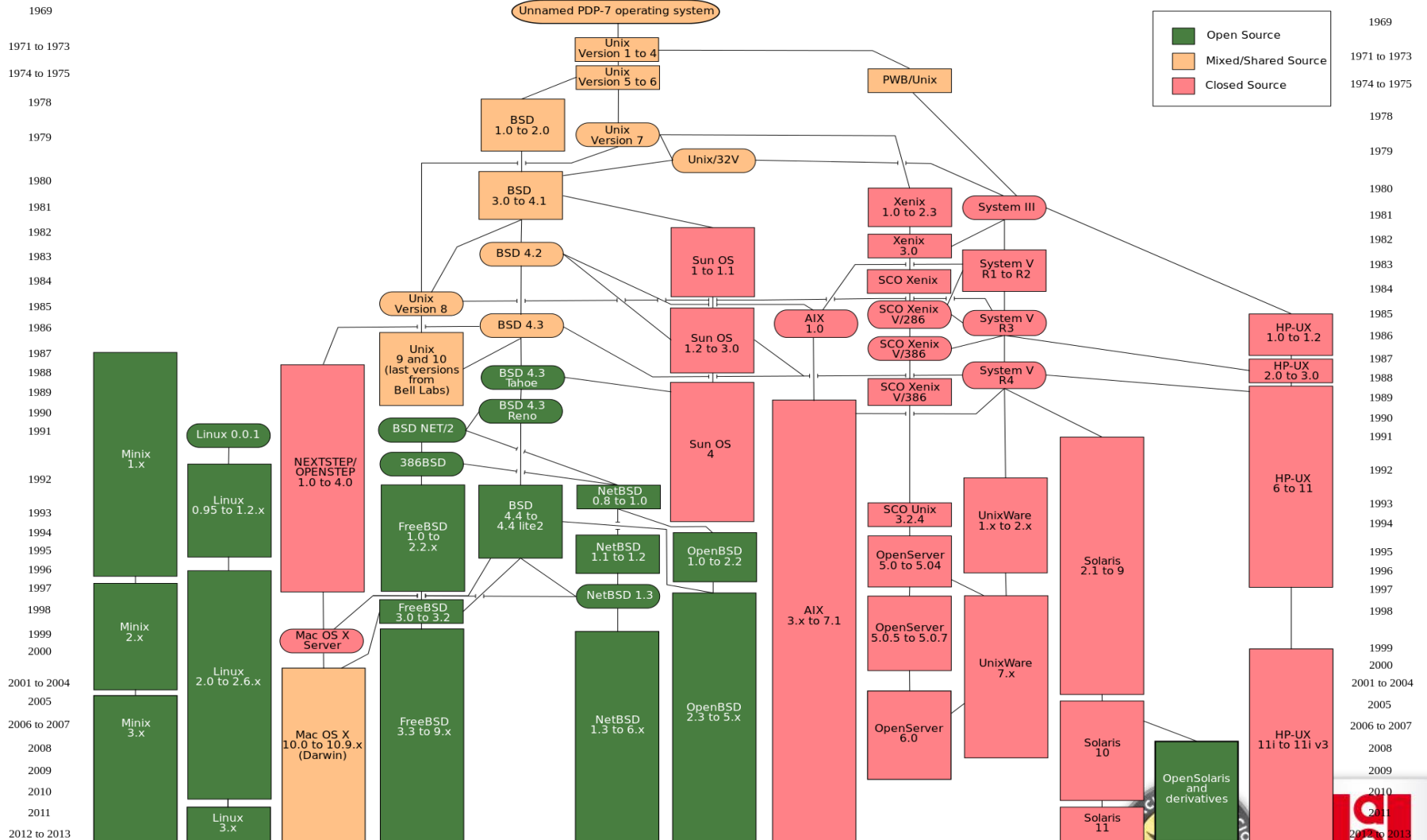


Kernel

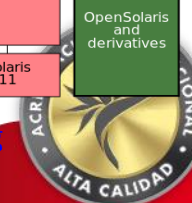


Historia de UNIX

www.uao.edu.co

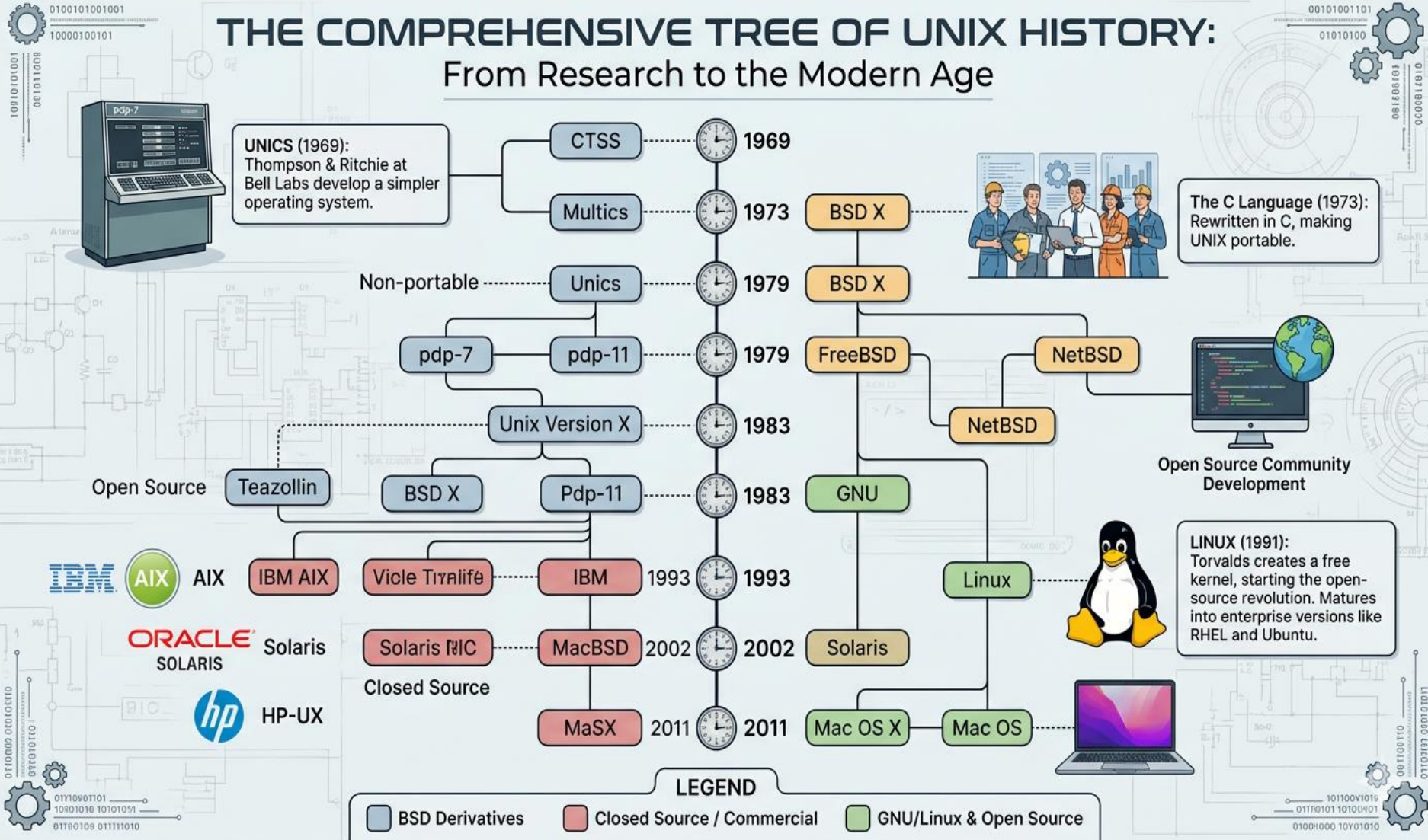


Fuente: http://upload.wikimedia.org/wikipedia/commons/5/50/Unix_history-simple.png



Historia de UNIX

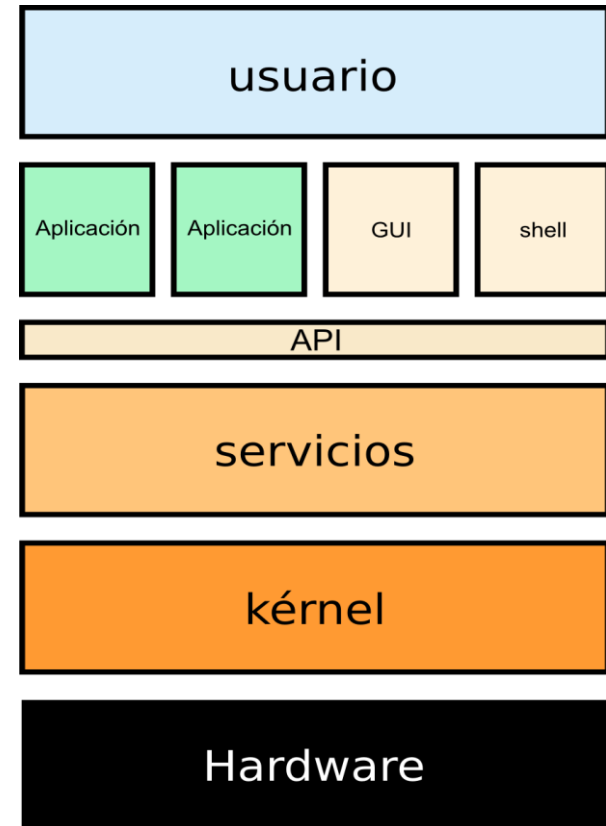
THE COMPREHENSIVE TREE OF UNIX HISTORY: From Research to the Modern Age



Universidad
AUTÓNOMA
de Occidente

Funciones del sistema operativo

- Un sistema operativo es una interfaz entre el hardware y el usuario y se encarga de gestionar y compartir los recursos.
- Las principales funciones del sistema operativo son:
 - Gestión de los recursos de la computadora
 - Ejecución de servicios para las aplicaciones
 - Ejecución de las órdenes de los usuarios





Kernel

- El núcleo de un sistema operativo es el Kernel, éste se encarga de administrar el hardware de la computadora. Por lo tanto es importante compilar un kernel estable, el cual se traduce en un desempeño óptimo y confiable del SO.
- Otro factor que influye en el desempeño del SO es el Sistema de Archivos, Linux posee gran variedad de Sistemas de Archivos que podemos utilizar, cada uno con sus características particulares.





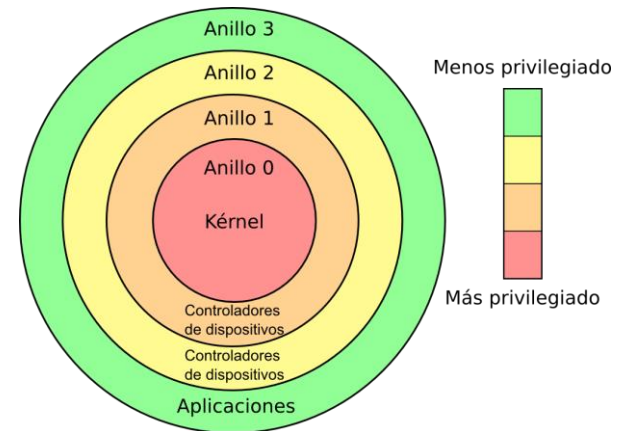
Funciones del kernel

- El kernel (núcleo) es la parte fundamental del sistema operativo y se encarga de manejar los recursos y permitir que los programas hagan uso de los mismos, siendo los principales recursos:
 - CPU
 - Memoria
 - Dispositivos de Entrada/salida
- Además el kernel es el encargado proporcionar:
 - Protección mediante diferentes niveles de acceso
 - Acceso compartido (multiplexado) a los recursos



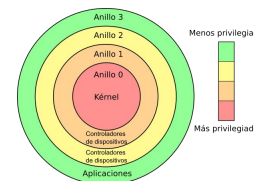
Niveles de seguridad

- Algunas CPU incluyen diferentes niveles de acceso, que se conocen como anillos (*rings*).
- Los diferentes kernel suelen utilizar al menos dos niveles para acceder tanto a la CPU como a la memoria:
 - kernel mode.
 - user mode
- KML: kernel mode linux
- FUSE: Filesystem in User Space (sshfs, ntfs-3g, ...)
- CUSE: Character devices in User Space





Niveles de seguridad



- KML: kernel mode linux
 - El Modo Kernel de Linux (KML) es una tecnología que permite la ejecución de programas comunes en el espacio de usuario dentro del espacio del kernel.
- FUSE: Filesystem in User Space (sshfs, ntfs-3g, ...)
 - FUSE (Filesystem in Userspace) es una interfaz del kernel de Linux que permite a los usuarios crear sistemas de archivos personalizados y montarlos sin necesidad de privilegios de superusuario o de modificar el código del kernel, a través de un módulo del kernel y una biblioteca de espacio de usuario.
- CUSE: Character devices in User Space
 - Un dispositivo de caracteres en el espacio del usuario en Linux es un dispositivo físico o virtual, como un puerto serie o un pseudoterminal, que se representa como un archivo especial en el directorio /dev y se accede mediante llamadas convencionales al sistema de acceso a archivos (como open, read, write). Estos dispositivos permiten a los programas de espacio de usuario interactuar directamente con el hardware o funciones del sistema a través de una interfaz de archivo.

Kernel Panic

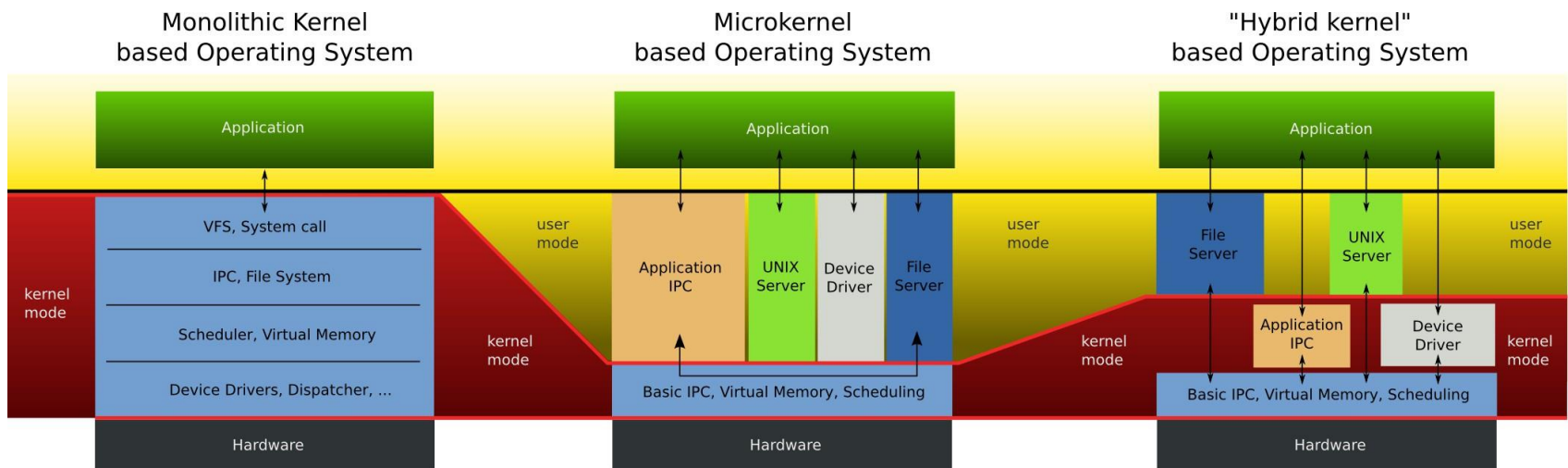
En Linux, un “panic” es un error insalvable del sistema detectado por el núcleo en oposición a los errores similares detectados en el código del espacio de usuario.

Es posible para el código del núcleo indicar estas condiciones mediante una llamada a la función de pánico situada en el archivo header `sys/system.h`. Sin embargo, la mayoría de las alertas son el resultado de excepciones en el código del núcleo que el procesador no puede manejar, como referencias a direcciones de memorias inválidas.

Generalmente esto es indicador de la existencia de un bug en algún lugar de la cadena de alerta. También pueden indicar una falla en el hardware como una falla del RAM o errores en las funciones aritméticas en el procesador, o por un error en el software.



Tipos de kernel



Fuente: <http://upload.wikimedia.org/wikipedia/commons/d/d0/OS-structure2.svg>

Principales arquitecturas CPU/ports

	Microsoft	Apple	Linux	Symbian	FreeBSD
x86	✓	✓	✓	✗	✓
alpha	✗	✗	✓	✗	✓
arm	✓*	✓*	✓	✓	✓
ia64	✓	✗	✓	✗	✓
m68k	✗	✓*	✓	✗	✗
mips	?	✗	✓	✗	✓
parisc	✗	✗	✓	✗	✗
sparc	✗	✗	✓	✗	✗
sparc64	✗	✗	✓	✗	✓
powerpc	✗	✓	✓	✗	✓
x86_64	✓	✓	✓	✗	✓



Principales sistemas operativos

Nombre	Creador	Basado en	Licencia
AIX	IBM	System-V	Privativa
FreeBSD	Comunidad	386BSD	BSD
HP-UX	HP	Unix	Privativa
GNU/Linux	Comunidad	Unix	GPL
Mac OS	Apple		Privativa*
NetBSD	Comunidad	386BSD	BSD
OpenSolaris	Sun	Solaris	CDDL
Windows server	Microsoft		Privativa
Windows	Microsoft		Privativa
ReactOS	Comunidad	Windows NT	GPL
z/OS	IBM	OS/390	Privativa

Fuente: http://en.wikipedia.org/wiki/Comparison_of_operating_systems





La lucha de los sistemas integrados

- Microprocesadores
 - ARM
 - MIPS
 - Intel
 - ...
- Dispositivos
 - mobile phone
 - PDA
 - smartphone
 - smartbook
 - netbook
 - ...
- Sistemas operativos
 - Blackberry
 - Symbian
 - Windows Mobile, Windows 7?
 - iPhone OS
 - Android
 - LiMo
 - Maemo
 - OpenMoko
 - Palm webOS
 - ...





Configuración del kernel

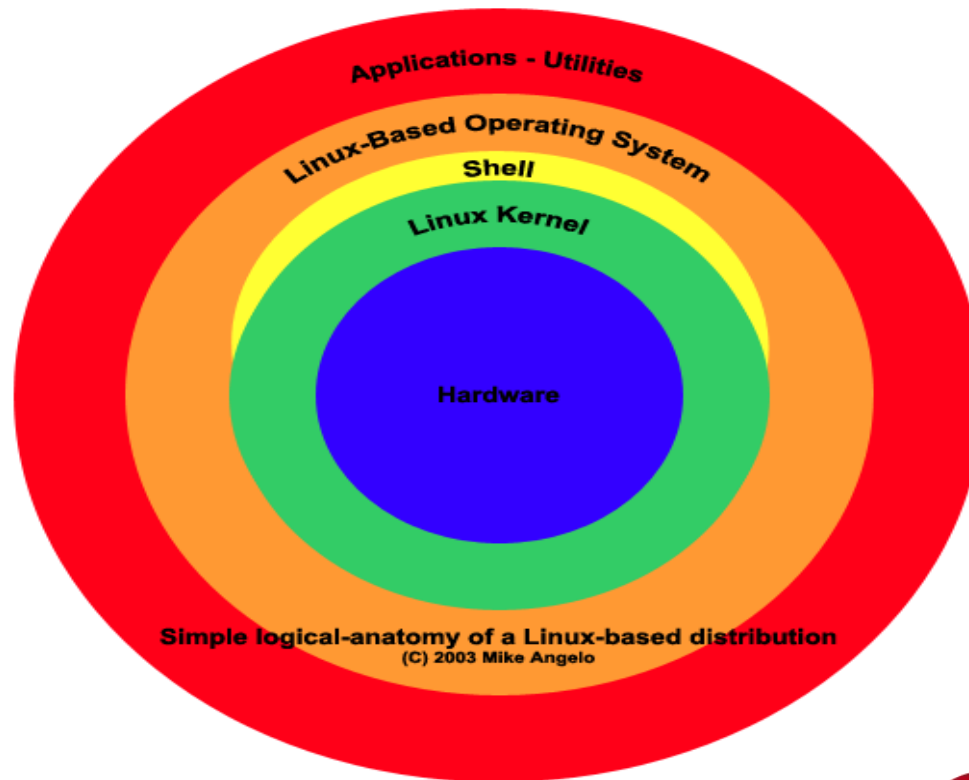
¿Qué es el núcleo?

- El kernel o núcleo de Linux se podría definir como el corazón de este sistema operativo. Es el encargado de que el software y el hardware de tu computadora puedan trabajar juntos.
- Las funciones mas importantes del mismo, aunque no las únicas, son:
 - Administración de la memoria, para todos los programas en ejecución.
 - Administración del tiempo de procesador, que estos programas en ejecución utilizan.
 - Es el encargado de que podamos acceder a los periféricos / elementos de nuestro computadora de una manera



Kernel

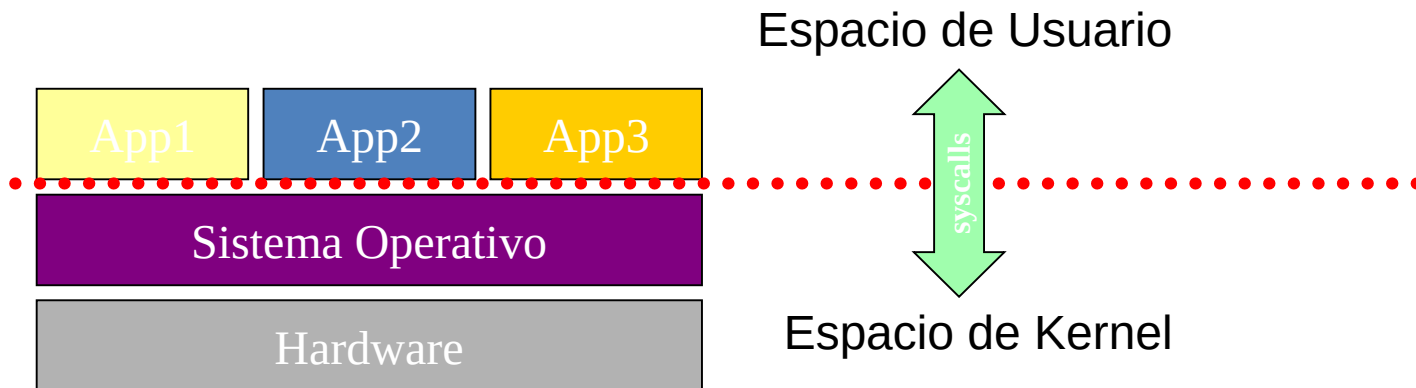
El Kernel se encarga de decidir qué programa podrá hacer uso de un dispositivo de hardware y durante cuanto tiempo, lo que se conoce como multiplexado, implementando las abstracciones de hardware.





kernel

Espacios





El kernel linux

- Características principales
- Características de la compilación
- Carga del sistema
- Manejo de módulos
- Compilación de módulos
- Compilación del kernel



El kernel & la shell

El kernel

- El término kernel se refiere al software que gestiona y asigna los recursos de la computadora. Es el corazón del sistema operativo (Kerris, 2010)
- La presencia del kernel simplifica en gran medida la escritura y uso de otros programas
- El kernel brinda al programador un enorme poder y flexibilidad al brindarle una capa de software para gestionar los recursos limitados de una computadora

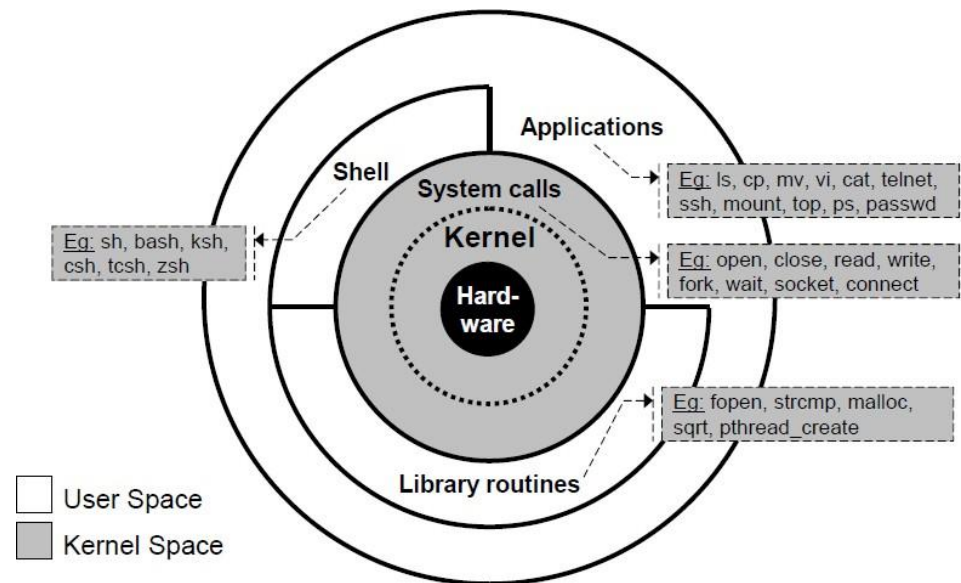


Imagen extraída de: https://1.bp.blogspot.com/-4zQoRo6_9i8/T8Ubi4GJT3I/AAAAAAAAACHl/SrBKJOWtg9E/s1600/aaa.png

El kernel & la shell

Tareas realizadas por el kernel

- Planificación de procesos
- Gestión de memoria
- Provisión de sistema de archivos
- Creación y terminación de procesos
- Acceso a dispositivos
- Manejo de la red
- Provisión de las llamadas al sistema (API)

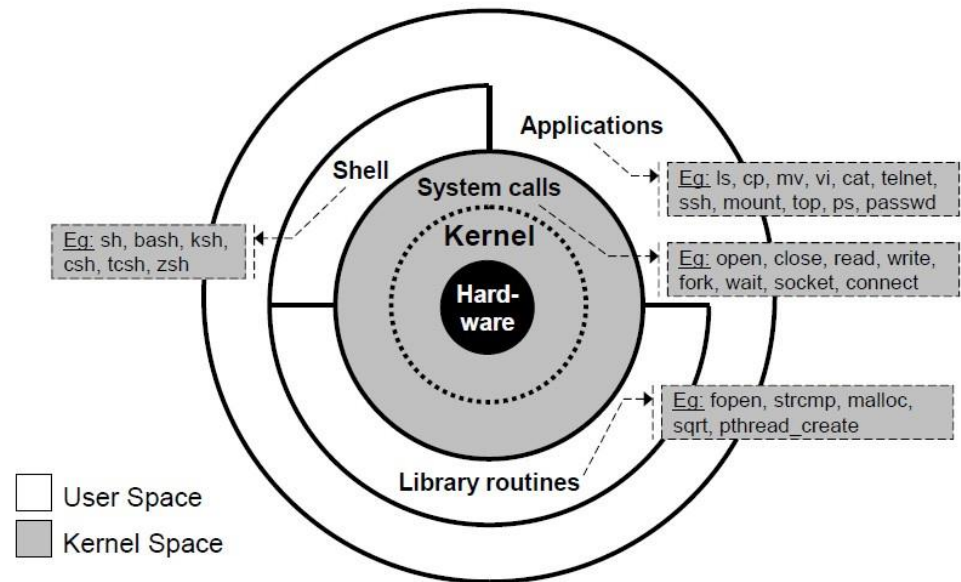
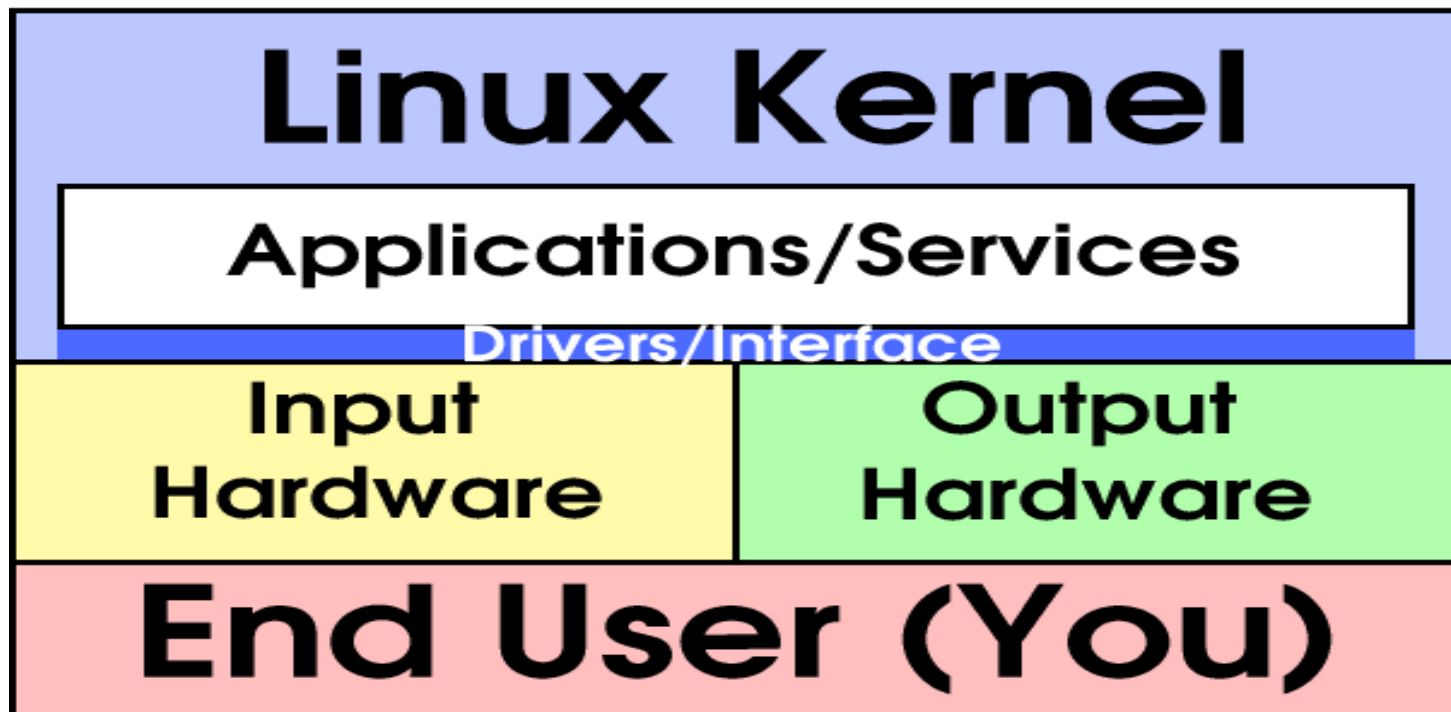


Imagen extraída de: https://1.bp.blogspot.com/-4zQoRo6_9i8/T8Ubi4GJT3I/AAAAAAAAACHl/SrBKjOWtg9E/s1600/aaa.png

**An Example Model
For a Linux OS
(Not Authoritative: Many possible models)**



Fuente: <http://mike.passwall.com/nblog/kernel-talk/kernel-model1.png>





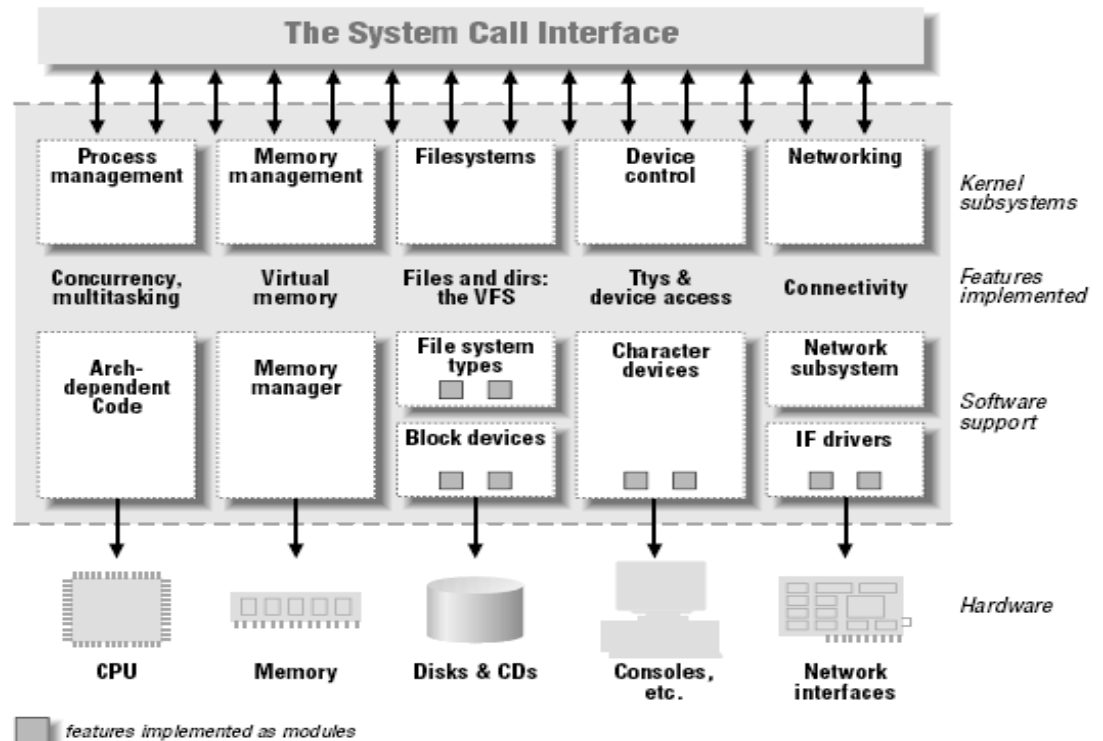
Clasificación y estructura

- Núcleo (kernel)
 - El núcleo o kernel es la parte fundamental del SO.
 - Es el código responsable de controlar y administrar los servicios y peticiones de recursos y hardware con respecto a uno o varios procesos.
- Para esquematizar las estrategias de diseño se suele mostrar el SO como una serie de niveles partiendo del hecho de que el kernel esta directamente en contacto con el hardware.



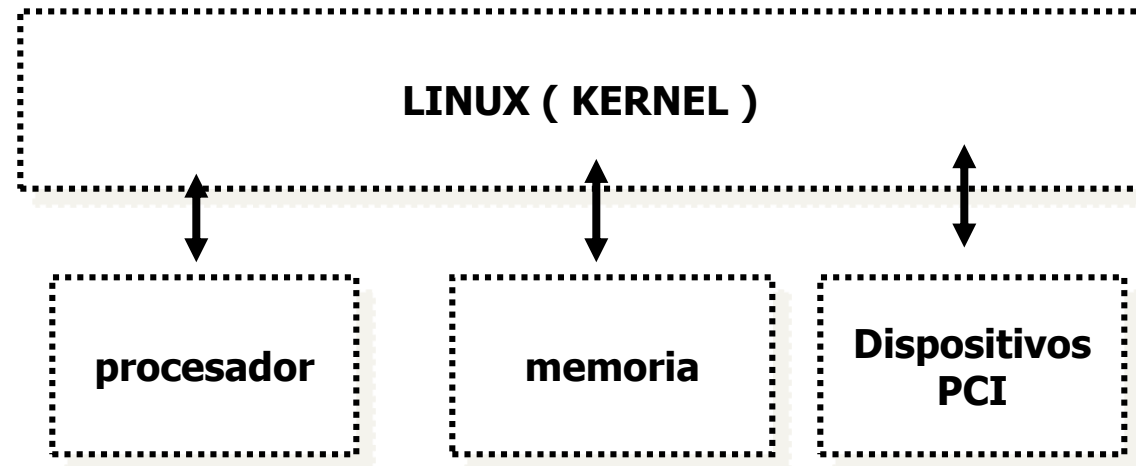
(Unix/Linux)

- Vista particionada del kernel
 - Tomado de Linux device drivers 2ª Ed.



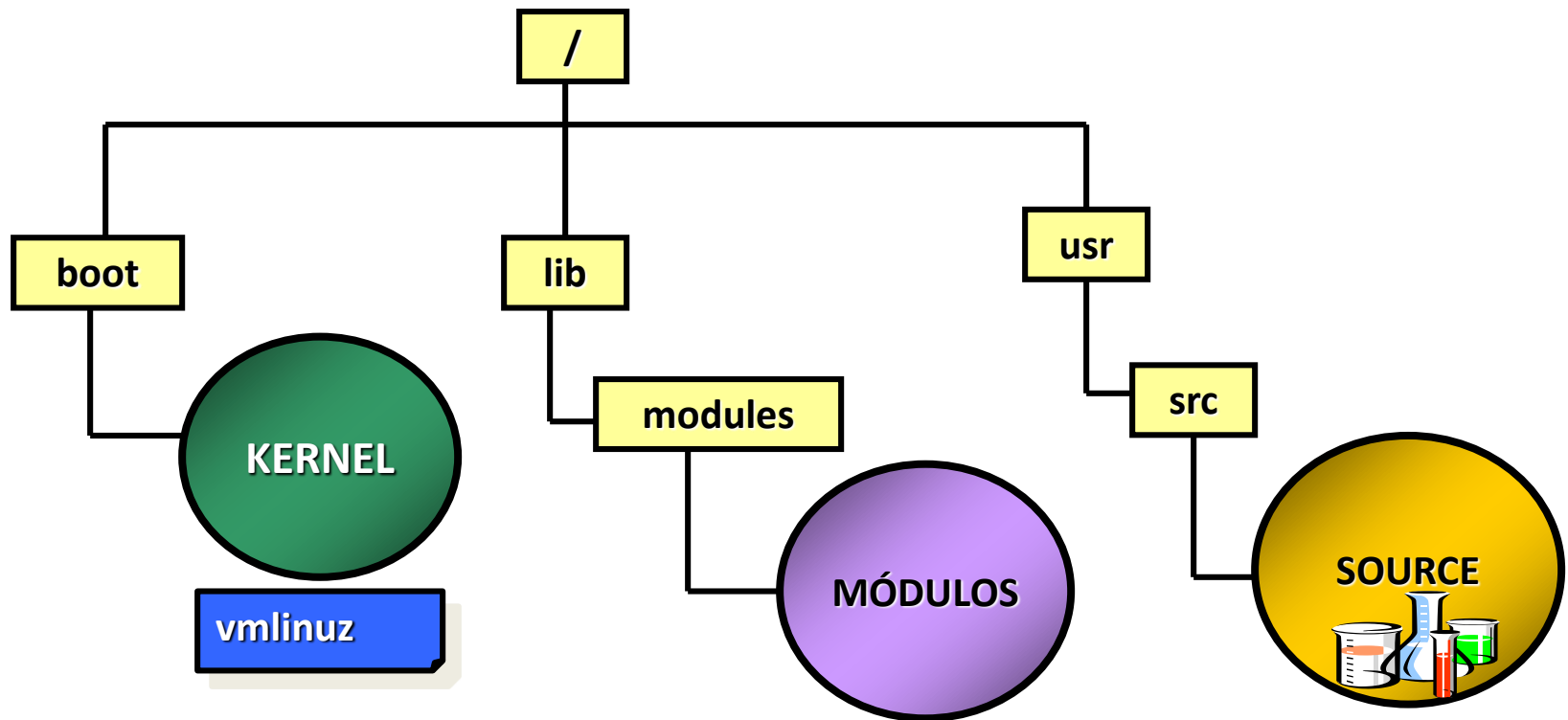


Información de Hardware



- Los archivos de información del sistema se ubican en el directorio /proc

Ubicación de elementos del Kernel



- Ubicación del Kernel, módulos y fuentes del Kernel.



Configuración del kernel

¿Qué es el núcleo? (II)

- Existen dos versiones del Linux kernel:
 - *Versión de producción*: La versión de producción, es la versión estable hasta el momento. Esta versión es el resultado final de las versiones de desarrollo o experimentales. Cuando el equipo de desarrollo del kernel experimental, decide que ha conseguido un kernel estable y con la suficiente calidad, se lanza una nueva versión de producción o estable. Esta versión es la que se debería utilizar para un uso normal del sistema, ya que son las versiones consideradas más estables y libres de fallos en el momento de su lanzamiento.
 - *Versión de desarrollo*: Esta versión es experimental y es la que utilizan los desarrolladores para programar, comprobar y verificar nuevas características, correcciones, etc. Estos núcleos suelen ser inestables y no se deberían usar, a no ser que sepas lo que haces.





Configuración del kernel

¿Qué es el núcleo? (III)

- Como interpretar los números de las versiones:
 - Las versiones del kernel se numeran con 3 números, de la siguiente forma: XX.YY.ZZ
 - XX: Indica la serie principal del kernel. Hasta el momento solo existen 6 versiones este numero cambia cuando la manera de funcionamiento del kernel ha sufrido un cambio muy importante.
 - YY: Indica si la versión es de desarrollo o de producción. Un número impar, significa que es de desarrollo, uno par, que es de producción.
 - ZZ: Indica nuevas revisiones dentro de una versión, en las que lo único que se ha modificado, son fallos de programación / bugs.





Configuración del kernel

La versión del núcleo Linux actualmente consta de cuatro números. Por ejemplo, asumamos que el número de la versión está compuesta de esta forma: A.B.C[D] (ej.: 2.2.1, 2.4.13 ó 2.6.12.3)

El **número A** denota la versión del núcleo

El **número B** denota la subversión del núcleo. Los números pares indicaban la versión “estable” lanzada, en cambio los números impares son versiones de desarrollo.

El **número C** indica una revisión mayor en el núcleo. Solo es cambiado cuando se introducen nuevos drivers o características; cambios menores se reflejan en el número D.

El **número D** se produce cuando un grave error requiere de un arreglo inmediato a nivel de Bug-fixes y parches de seguridad.





Configuración del kernel

¿Qué es el núcleo? (IV)

- Unos ejemplos nos ayudaran a entenderlo mejor:
 - ej1: version del kernel 2.4.0: Kernel de la serie 2 (XX=2), version de produccion 4 (YY=4 par), primera version de 2.4 (ZZ=0).
 - ej2: version del kernel 2.4.1: Kernel de la serie 2, version 4, en el que se han corregido errores de programación presentes en la version 2.4.0 (ZZ=1).
 - ej3: version del kernel 2.5.0: version 0 del kernel de desarrollo 2.5.





Carga del sistema - I

- Se inicia el sistema cargando la BIOS
- Se realiza la secuencia POST (*Power-On Self-Test*)
- Se lee el MBR del disco duro donde está la información de las particiones del disco y el gestor de arranque (normalmente GRUB).
- Se carga en memoria el Archivo ejecutable comprimido *vmlinux-...*
- Se monta el initramfs (Archivo *initrd-...*) ← opcional
- Se comprueba la memoria, tipo de placa y CPU(s)
- Se activa el sistema Plug and Play
- Se inicializan los dispositivos virtuales (LVM y RAID)
- Se libera la memoria ocupada por el initramfs
- Se ejecuta el proceso *init* con PID=1
- Se ejecutan los scripts de */etc/rcS.d* ← depende de la distro





Carga del sistema - II

- Se establece el nombre del equipo (*hostname*)
- Se monta VFS
- Se inicia udevd, que puebla */dev* y carga los módulos necesarios
- Se chequea el sistema de archivos raíz
- Se procesa el Archivo */etc/modules*
- Se chequean todos los sistemas de archivos
- Se cargan los parámetros del kernel especificados en */etc/sysctl.conf*
- Se montan todos los sistemas de archivos
- Se limpian los archivos temporales */tmp*, */var/run* y */var/lock*
- Se levantan las interfaces de red
- Se ejecutan en orden los scripts del resto de niveles de ejecución





Manejo de módulos

La mayoría de los módulos se cargan automáticamente cuando es necesario, pero es posible cargarlos o descargarlos manualmente:

- *lsmod*: Lista los módulos cargados
- *modprobe* <módulo>: Carga el módulo en memoria
- *modprobe -r* <módulo>: Descarga el módulo de la memoria
- *modprobe -l*: Muestra la lista de módulos disponibles
- *modinfo* <módulo>: Muestra la información del módulo
- *depmod*: Actualiza las dependencias de los módulos



Configuración del kernel

¿Por qué puedo necesitar actualizar el núcleo?

- Los núcleos nuevos normalmente ofrecen la posibilidad de entenderse con más accesorios hardware (o sea, incluyen más manejadores), se ejecutan más rápidamente, son más estables o corrigen errores de otras versiones. Mucha gente se actualiza el núcleo para poder usar nuevos manejadores que necesitan o librarse de ``bugs'' de la versión que usaban.





Características principales

- Kernel del sistema GNU/Linux, licenciado bajo la GNU GPL
- Desarrollo colaborativo de miles de personas
- Monolítico
- LKM: Loadable kernel module
- Última versión estable: 6.8
 - Kernel.org
- Soporte para virtualización
- Portado a gran cantidad de arquitecturas, desde pequeños dispositivos a grandes supercomputadoras.





Características de la compilación

- El código fuente de rama oficial del núcleo ocupa actualmente 404MiB !!!
- Los componentes del kernel se compilan de dos formas:
 - Se incluyen dentro de un Archivo ejecutable enlazado estáticamente y que habitualmente se denomina *vmlinux* o *zImage*
 - Se compilan individualmente en archivos objetos con extensión *.ko* que se cargan en memoria a demanda (están ubicados en */lib/modules*)
- Soluciones para hardware no detectado en el arranque:
 - Se aumenta el tamaño del Archivo ejecutable (*bzImage* o *big zImage*)
 - Se montan temporalmente algunos módulos en memoria (*initramfs*)
- Distribuciones de uso general en sistemas x86:
 - Enorme variedad de hardware
 - Se incluyen gran cantidad de módulos
- Es posible compilar un kernel para un hardware determinado y reducir mucho su tamaño.





El kernel de FreeBSD es el corazón del sistema operativo. Es el responsable de la gestión y control de la memoria, reforzamiento de los controles de seguridad, gestión de redes, acceso a dispositivos y muchas cosas más. FreeBSD es un sistema dinámico y configurable, pero a veces sigue siendo necesario reconfigurar y recompilar un kernel hecho a la medida de nuestras necesidades.

Compilar un kernel personalizado es uno de los ritos de iniciación más importantes a los que los usuarios de casi todos los BSD han de enfrentarse. Este proceso lleva su tiempo, pero le aportará grandes beneficios a su sistema FreeBSD. A diferencia del kernel GENERIC, con el que puede funcionar una cantidad apabullante de hardware disponible en el mercado, un kernel personalizado contiene únicamente lo necesario para que funcione el hardware de su sistema.

- Mayor rapidez en el arranque del sistema. Dado que el kernel sólo tiene que probar el hardware que realmente está en el sistema el tiempo que necesitará para arrancar se reducirá visiblemente.
- Menor uso de memoria. Generalmente un kernel personalizado utiliza menos memoria que un kernel GENERIC, lo cual es importante ya que el kernel debe encontrarse siempre en memoria real. Por ésta razón un kernel personalizado puede ser de gran utilidad en sistemas con una cantidad limitada de memoria RAM.
- Hacer funcionar hardware específico. Un kernel personalizado le permite añadir al sistema dispositivos como tarjetas de sonido, que no están incluidas en el kernel GENERIC.





Antes de compilar un kernel personalizado conviene pensar antes los motivos. Si lo que se necesita es añadir hardware específico al sistema es posible que sea posible utilizarlo mediante un módulo.

Los módulos del kernel están en el directorio `/boot/kernel` y se pueden cargar dinámicamente en un kernel en funcionamiento con `kldload(8)`. La mayoría, por no decir todos, los controladores del kernel tienen un módulo específico y una página de manual. Por ejemplo, en la sección anterior hablábamos del controlador Ethernet inalámbrico `ath`. La página de manual de este dispositivo contiene la siguiente información:

- Antes de compilar un kernel personalizado conviene pensar antes los motivos. Si lo que se necesita es añadir hardware específico al sistema es posible que sea posible utilizarlo mediante un módulo.
Alternatively, to load the driver as a module at boot time, place the following line in `loader.conf(5)`:
`if_ath_load="YES"`
- Los módulos del kernel están en el directorio `/boot/kernel` y se pueden cargar dinámicamente en un kernel en funcionamiento con `kldload(8)`. La mayoría, por no decir todos, los controladores del kernel tienen un módulo específico y una página de manual. Por ejemplo, en la sección anterior hablábamos del controlador Ethernet inalámbrico `ath`. La página de manual de este dispositivo contiene la siguiente información:



la compilación del kernel. Todos los directorios mencionados hacen referencia al directorio principal, que es `/usr/src/sys`, al que también podemos acceder desde `/sys`.

la arquitectura del sistema, por lo que puede ser i386, alpha, amd64, ia64, powerpc, sparc64 o pc98 (una arquitectura alternativa, similar a PC y muy popular en Japón). Todo lo que existe dentro de un directorio de una arquitectura en particular, es específico para dicha arquitectura; el resto del código es común para todas las plataformas en las que FreeBSD puede funcionar.

Si no existe el directorio `/usr/src/sys` en su sistema quiere decir que no están instaladas las fuentes del kernel. La forma más sencilla de tenerlas es mediante `/stand/sysinstall`. Como usuario root seleccione Configure, luego Distributions, después src y finalmente sys.





Ejemplo de Archivo de configuración de un kernel GENERIC al que se han añadido comentarios adicionales donde se ha visto que era necesario abundar un poco para mayor claridad. Este ejemplo es igual (o, en el peor de los casos, casi igual) que la copia del mismo que tiene usted en `/usr/src/sys/i386/conf/GENERIC`.

```
machine      i386
```

Esta es la arquitectura de la máquina. Debe ser alpha, amd64, i386, ia64, pc98, powerpc o sparc64.

```
cpu          I486_CPU
```

```
cpu          I586_CPU
```

```
cpu          I686_CPU
```

Las opciones anteriores definen el tipo de CPU que haya en su sistema. Puede dejar varias líneas de CPU (si, por ejemplo, no está seguro de usar I586_CPU o I686_CPU), pero si está personalizando su kernel es mucho mejor que solamente ponga la CPU que tenga.

- `cat /etc/os-release`
- `freebsd-version -u -k -r`
- `getconf LONG_BIT`



ACTIVIDADES

Configuring the FreeBSD Kernel

```
# cd /usr/src/sys/i386/conf  
# mkdir /root/kernels  
# cp GENERIC /root/kernels/CLASEKERN  
# ln -s /root/kernels/CLASEKERN
```

Building and Installing a Custom Kernel

```
cd /usr/src  
  
# make buildkernel KERNCONF=CLASEKERN  
  
# make installkernel KERNCONF=CLASEKERN  
  
init 6
```





Trabajo Investigativo

Entrega individual Octubre 5 2026

- Presentar un sistemas operativo en que su kernel es:

kernel monolítico.

Kernel con diseño modular.

Sistemas estructurados en capas

Sistemas con micronúcleo

Máquina virtual

Que tipo de Kernel tienen los siguientes SO

(BeOS, Mach, Mac OS X, newOS).

(AdeOS, Eros, KeyKOS, Brix-OS).

(Opal, Mungi, BriX). Single address space operating system

(MIT exokernel).

Presentar versión 6 de Octubre 19 2026*****

- Instalar nuevo Kernel en ORACCLELINUX UBUNTO o CENTOS





www.uao.edu.co

